Compito 2.1 APA

Fabio Nicolosi s7592267@studenti.unige.it

May 2026

1 Introduzione

L'obiettivo di questo compito era studiare il comportamento probabilistico di algoritmi di ordinamento randomizzati. In particolare, sono stati analizzati due tipi di algoritmi randomizzati: *Las Vegas* e *Monte Carlo*. L'algoritmo Las Vegas introduce un elemento casuale al fine di evitare i casi peggiori deterministici: un algoritmo deterministico, infatti, è vulnerabile a sequenze di input costruite appositamente per degradare le prestazioni. L'algoritmo Monte Carlo, invece, pone un limite al numero di operazioni eseguite, garantendo così un tempo di esecuzione controllato a scapito di una piccola probabilità di fallimento. Come algoritmo di base è stato scelto il QuickSort. Nella prima fase è stato implementato e analizzato **LVQuickSort**, che utilizza un pivot scelto casualmente. Successivamente, sfruttando la disuguaglianza di Markov — che permette di quantificare la probabilità che una variabile casuale superi una certa soglia — è stato implementato **MCQuickSort**, che esegue **LVQuickSort** al più k volte, interrompendo ogni esecuzione se il numero di confronti supera il limite $2\hat{\mu}$ e restituendo successo alla prima run completata entro tale limite.

```python
def quicksortLV_stop(lst: list, s: int, f: int, count: list, limit:int):
    if s >= f:
        return True

    if count[0] >= limit:
        return False

    # Pivot casuale
    pivot_idx = random.randint(s, f)
    pivot_val = lst[pivot_idx]

    # Sposta il pivot alla fine
    lst[pivot_idx], lst[f] = lst[f], lst[pivot_idx]

    # Partizione in-place (Lomuto)
    store = s
    for i in range(s, f):
        count[0] += 1
        if lst[i] <= pivot_val:
            lst[i], lst[store] = lst[store], lst[i]
            store += 1

    # Rimetti il pivot al posto giusto
    lst[store], lst[f] = lst[f], lst[store]

    return (quicksortLV_stop(lst, s, store - 1, count, limit) and quicksortLV_stop(lst, store + 1, f, count, limit));


def quicksortMC(listN:list[int], k:int):

    for _ in range(k):
        count = [0]
        listNum = [random.uniform(MIN, MAX) for _ in range(N_ELEM)]

        if quicksortLV_stop(listN, 0, N_ELEM-1, count, (2*mediaEmp)):
            return True

    return False
```

Implementazione Monte Carlo

2 LVQuickSort

Input: S, sequenza di numeri
Output: S, sequenza ordinata dal più piccolo al più grande

`if` $|S| \leq 1$
  `then return` S
`else`
  1. campiona s da S con probabilità uniforme
  2. forma $S_<$ e $S_\geq$ confrontando ogni numero di $S \setminus \{s\}$ con s
  3. *LVQuickSort*($S_\geq$)
  4. *LVQuickSort*($S_<$)
  5. `return` la concatenazione di $S_<$, s e $S_\geq$

Pseudo codice LVquickShort

2.1 Spiegazione algoritmo e pivot casuale

Come algoritmo di base è stato scelto **LVQuickSort**, in quanto rappresenta un buon esempio di algoritmo randomizzato di tipo *Las Vegas*: fornisce sempre un output corretto, ma non garantisce un tempo di esecuzione fisso, poiché esso non dipende più dall'input. Questo rende impossibile costruire una sequenza che forzi sistematicamente il caso peggiore $\theta(n^2)$. Di conseguenza, il tempo di esecuzione diventa **una variabile** casuale, con caso peggiore $\theta(n^2)$ e caso medio $\theta(n \log n)$.

2.2 Disuguaglianza di Markov

La *disuguaglianza di Markov* fornisce un limite superiore alla probabilità che una variabile casuale **non negativa** assuma valori molto più grandi della sua media.

In sintesi: è improbabile che il numero di confronti assuma valori molto superiori al valore atteso.

2.2.1 Formula

Per ogni $a > 0$, la probabilità che $X \geq a$ è limitata superiormente dal rapporto tra il valore atteso $\mathbb{E}[X]$ e la soglia a:

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

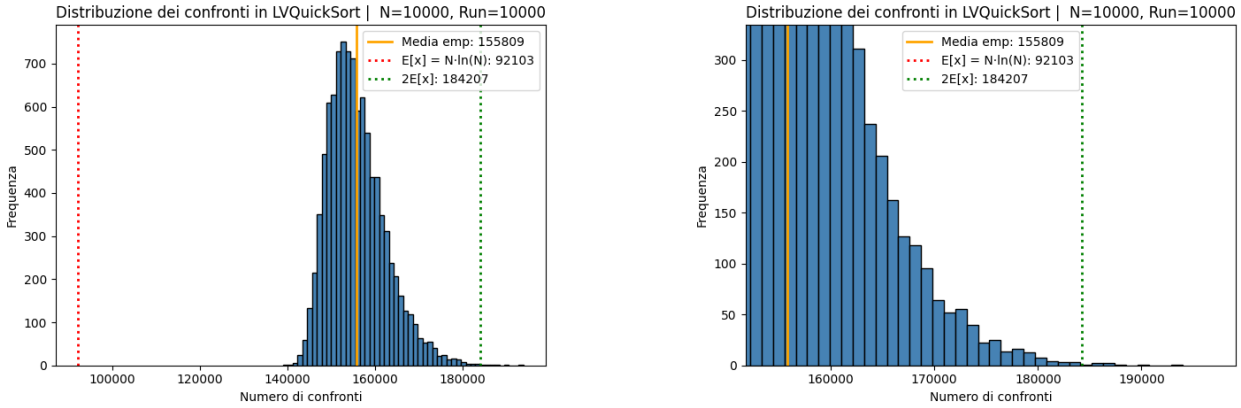
2.2.2 Interpretazione pratica

Scegliendo come soglia il doppio del valore atteso, ovvero $a = \mathbb{E}[X]$,la formula diventa:

$$\mathbb{P}(X \geq 2\mathbb{E}[X]) \leq \frac{1}{2}$$

significa che, per qualsiasi distribuzione non negativa, la probabilità che la variabile superi il doppio della sua media è al massimo del 50%.

2.3 Pessimismo Markov



Confronto delle distribuzioni.

Come si osserva nei grafici, la linea verde rappresenta il limite $2\mathbb{E}[X]$ imposto dalla disuguaglianza di Markov. Si nota che pochissimi elementi *(8 su 10.000 run)* superano effettivamente tale soglia, a differenza di quanto garantito dalla disuguaglianza, che prevede un limite superiore del 50% Questo dimostra quanto la stima di Markov sia pessimista: nella pratica la distribuzione è molto più concentrata attorno alla media empirica

```
Media empirica:  155809.35
E[X]  = N*ln(N):    92103.40
2E[X] formula(3) = 2*N*ln(N):  184206.81
Elementi sopra limite Markov: 8
PMF empirica (Formula 4): 0.00080
```

Dati

3 MCQuickSort

```python
def quicksortMC(listN:list[int], k:int):

    for _ in range(k):
        count = [0]
        listNum = [random.uniform(MIN, MAX) for _ in range(N_ELEM)]

        if quicksortLV_stop(listN, 0, N_ELEM-1, count, (2*mediaEmp)):
            return True

    return False


for _ in range(N_RUN):
    if quicksortMC (listNum, 3) :
        Ok[0] +=1
    else:
        notOk[0] +=1
```

Utilizzo MC

L'idea alla base di **MCQuickSort(S, k)** è quella di eseguire al più k volte *LVQuickSort*, interrompendo ogni esecuzione nel momento in cui il numero di confronti supera $2\hat{\mu}$ (dove $\hat{\mu}$ la media empirica calcolata su 10^4 *run*, usata come stima di $\mathbb{E}[X]$).
Alla prima run che completa l'ordinamento restando entro il limite superiore, MCQuickSort restituisce il risultato. Al contrario, se tutte le k esecuzioni vengono interrotte prima di completare l'ordinamento, MCQuickSort fallisce. In questo modo si garantisce un tempo di esecuzione limitato a $\theta(n \log n)$, a scapito di una piccola probabilità di fallimento.

3.1 Confronto delle due formule

La formula (3) deriva dalla disuguaglianza di Markov — e come si osserva dai grafici — rappresenta una garanzia teorica pessimista: Markov non utilizza alcuna informazione sulla distribuzione di X ma solo il suo valore atteso. Il limite $\frac{1}{2}$ è quindi un tetto superiore, non la probabilità reale.
La formula (4) sostituisce $\mathbb{E}[X]$ con la media empirica $\hat{\mu}$, calcolata su 10^4 *run*,utilizzando la distribuzione sperimentale come approssimazione della PMF vera:

$$\mathbb{P}(X \geq 2\mathbb{E}[X]) \leq \frac{1}{2}$$

con $\epsilon = 10^{-5}$,ovvero quasi **ZERO**. Nei risultati sperimentali, su 10^4 *run* solo 8 elementi superano la soglia $2\hat{\mu}$, confermando che la probabilità reale è circa $\frac{8}{10^4} = 0,0008$ — molto più bassa del 50% garantito dalla formula (3).

4 Conclusioni

Come si è potuto osservare, la formula (4) risulta molto più precisa della (3), poiché sfrutta la conoscenza empirica della distribuzione: non ci si limita al solo valore atteso, ma si osserva direttamente quante esecuzioni superano la soglia $2\hat{\mu}$.Questo spiega anche perché **MCQuick-Sort(S, k)** con $k = 3$ funzioni così bene.
La probabilità di fallire in una singola run è $\epsilon \approx 0,0008$ (non $\frac{1}{2}$ come stimato da *Markov*). Poiché ogni run di *LVQuickSort* opera su una lista generata indipendentemente, gli eventi di fallimento sono indipendenti, e quindi la probabilità di fallire k volte consecutive è ϵ^k.Nel nostro caso:

$$\epsilon^3 \approx 5 * 10^{-10}$$

praticamente nulla.

```
Media empirica:  155809.35
E[X]  = N*ln(N):    92103.40
2E[X] formula(3) = 2*N*ln(N):  184206.81
Elementi sopra limite Markov: 8
PMF empirica (Formula 4): 0.00080
-------------MonteCarlo-------------
K= 3
-Probabilità successo: 1.0000
-Probabilità teorica (1 - 1/2^k): 0.8750
```

Dati-Completi

I risultati confermano questo: la probabilità di successo empirica è $1,0000$ (su 10^4 prove nessun fallimento registrato) mentre la stima teorica conservativa di $1 - \frac{1}{2^3} = 0,875 = 87,5\%$ risulta notevolmente più pessimista.

5 Utilizzo - IA

Poiché lo scopo di questo compito era comprendere e verificare nella pratica ciò che fino ad ora era rimasto solo teoria, ho ritenuto opportuno aggiungere un paragrafo trasparente sull'utilizzo degli strumenti che mi hanno supportato, nella speranza che questo possa essere utile anche per valutare non solo la comprensione dell'argomento, ma anche un uso responsabile dell'intelligenza artificiale.

5.1 Svolgimento del compito

Durante lo svolgimento del compito ho utilizzato l'IA in due occasioni distinte. In primo luogo, per assicurarmi di aver compreso correttamente le richieste del compito, ho chiesto a Gemini di spiegarmele con parole più semplici, pur avendo già una mia interpretazione iniziale [chat: Gemini-Link]. In secondo luogo, avendo riscontrato diverse difficoltà nell'utilizzo di Python — linguaggio che non avevo mai usato in precedenza — e volendo assicurarmi che il codice scritto fosse corretto e leggibile, ho utilizzato Claude come supporto per la revisione [chat: Cloude-Link]. Spero che questo utilizzo sia considerato corretto. Ritengo personalmente di aver compreso a fondo i concetti trattati e gli errori commessi durante il percorso. Sarei molto grato di ricevere un feedback, sia sul compito che sull'utilizzo dell'IA.